

IART 24/25 FEUP

Assignment 1

Baker's Dozen Solitaire: Heuristic Search Methods

Group A1_64:

Hugo Barbosa - 202205774

João Parada - 201405280

Luís Ganço - 202004196

Project Specification

- **Project Goals:**
 - Implement a GUI-based human-playable game.
 - Develop an AI solver using uninformed (DFS, Iterative Deepening) and informed (A*, WA*, Greedy) search algorithms.
 - Compare algorithm performance (solution quality, time, memory, states explored).
 - Allow a **Human game mode** where the user can interactively solve the game and request **hints** (next move) from the AI.
- **Game Rules:**
 - Build foundation piles (Ace to King) in ascending order by suit.
 - Tableau cards are built downward regardless of color.
 - Only **one** card can be moved at a time; empty tableau slots **cannot** be filled.



Problem Formulation as a Search Problem

State Representation

- **Tableau:** $\text{Matrix}[13][N]$, where $\text{tableau}[\text{col}][-1]$ is the top card of the column.
- **Foundation:** $\text{List}[4]$, containing only the last card added to the respective pile.
- **Move History:** $\text{List}[3]$ to track the last 3 moves (undo functionality)..

Initial State

- **Tableau:** $\text{Matrix}[13][4]$, deck shuffled and dealt into 13 columns of 4 cards each, with Kings moved to the bottom of their columns.
- **Foundation:** Empty list.
- **Move History:** Empty.

Problem Formulation as a Search Problem

Operators (Moves)

- **Move to Foundation:** Place the top card of a tableau column onto a foundation pile.
 - **Preconditions:** Foundation pile is empty, and card is an Ace; or the card follows the rank of the top foundation card in the same suit (e.g., 2 ♠ on Ace ♠).
 - **Effect:** Card is removed from the tableau and added to the corresponding foundation pile.
- **Move within Tableau:** Place the top card of one tableau column onto the top of another tableau column.
 - **Preconditions:** The moving card must be exactly one rank lower than the top card of the destination column, regardless of suit (e.g., 7 ♦ on 8 ♠).
 - **Effect:** Card is removed from the source column and placed on the destination column.
- **Cost Function:** Cost = 1 per move.

Goal State

- **Solved:** All 52 cards are placed in foundation piles, with each suit built from Ace to King.
- **Failure:** No valid moves remain and the tableau is stuck or time ran out.

Heuristic Functions

H1: Tableau Card Count

- Measures the number of cards left in the tableau. Fewer cards = closer to completion.
- **Goal:** Minimize remaining tableau cards for faster progress.

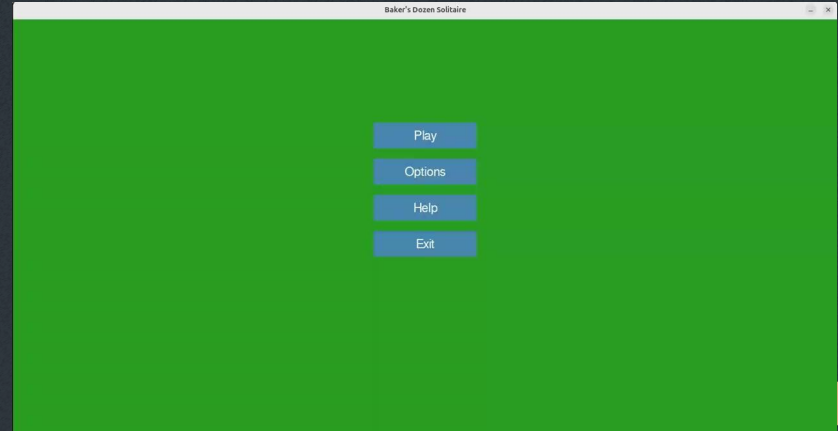
H2: Card Foundation Distance

- Calculates the "distance" of the cards in tableau piles from their foundation target (e.g., King = 0, Queen = 1).
- **Goal:** Reduce the total distance to foundation for quicker completion.

Implementation Work So Far (Checkpoint)

- **Programming Language:** Python
- **GUI Library:** PyGame (early UI prototype)
- **Data Structures:** Lists for foundations, matrix for tableau, list for last 3 moves performed

Next Steps: Implement the heuristics and search algorithms, refine GUI interactions, set up the experiment framework, and make the game work for variable card number (from 4 to 13 of each symbol), and variable time limit.



Uninformed Search Methods

- **Depth-First Search**

- Avoids cycles by checking previous unique states (visited nodes).
- Parameter “depth_limit”: Limits the maximum moves each solution can have (avoids surpassing Python recursion limit).
- Parameter “useless_moves”: Maximum non-progressing moves in a row to improve performance
- Does not move the same card twice in a row.
- We tested this algorithm with 3 different parameter combinations: DFS: depth_limit = 100, max_useless = 3, DFS1: depth_limit = 200, max_useless = 10, DFS2: depth_limit = 500, max_useless = 50.

- **Improved Depth-First Search**

- DFS with successor node ordering: Visits most improving nodes first, usually performs much better on simpler solutions.
- We tested this algorithm with the same 3 parameter combinations.

- **Iterative Deepening**

- Performs DFS with increasing “depth_limit”s.
- Very inefficient for Baker’s Dozen, since most solutions require an high number of moves.

Heuristic Functions (Revisited)

- After the checkpoint, we made the following changes to our heuristics:
- **Greedy Heuristic**
 - **Goal:** Fast progress towards foundations.
 - **Key Factors:**
 - Heavy penalty for remaining/blocked cards (x3).
 - Rewards foundation progress (x5).
 - Penalizes buried low cards (Ace, 2) and imbalanced foundations.
- **A* and Weighted A* Heuristic**
 - **Goal:** Balance optimality & efficiency.
 - **Key Factors:**
 - Focus on blocking ranks (ex: queen in 12-column games).
 - Bonus for empty columns (-5).
 - Penalty for broken sequences (+2).
 - Weighted A* adds scaling to prioritize heuristic.
 - We tested 3 different weights, $W=1.2$, $W=1.5$, $W=2$.

Heuristics Strategy Overview

Heuristic	Focus	Special Traits
Greedy	Fast progress to foundation	Penalizes imbalance & buried low cards
A*	Balanced exploration	Emphasizes blocking cards & sequences
WA*	Faster but less optimal pathfinding	Multiplies heuristic to prefer exploration

Implemented Algorithms

Greedy Search

- Always picks the **best** immediate move
- Fast but **suboptimal**

DFS & DFS Improved

- **Standard:** Depth-first with pruning
- **Improved:** Prioritizes reducing tableau size

Iterative Deepening

- DFS with increasing depth limits
- Not suitable for Baker's Dozen

A* Search

- Optimal ($f(n) = g(n) + h(n)$)
- Good for guaranteed solutions

Weighted A* Search

- Faster ($f(n) = g(n) + 1.5 \times h(n)$), for $w = 1.5$
- Good speed/solution balance

Algorithm	Time Complexity	Space Complexity
Greedy	$O(b \cdot h)$	$O(b)$
DFS	$O(b^d)$	$O(b \cdot d)$
Iterative Deepening	$O(b^d)$	$O(d)$
A*	$O(b^h)$	$O(b^h)$
WA*	$O(b^{(h/w)})$	$O(b^{(h/w)})$

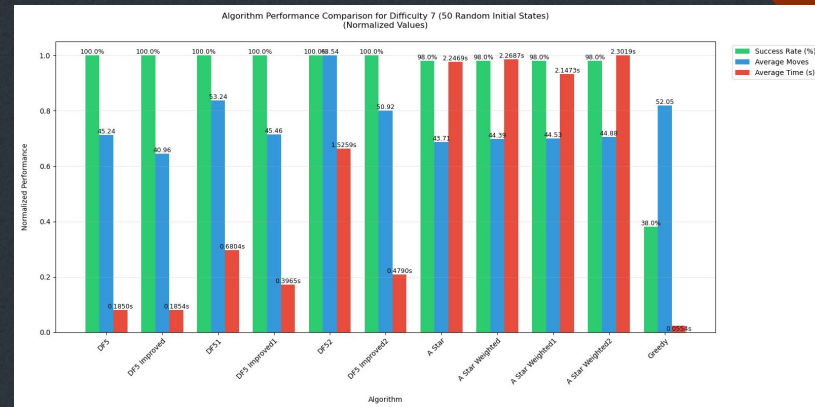
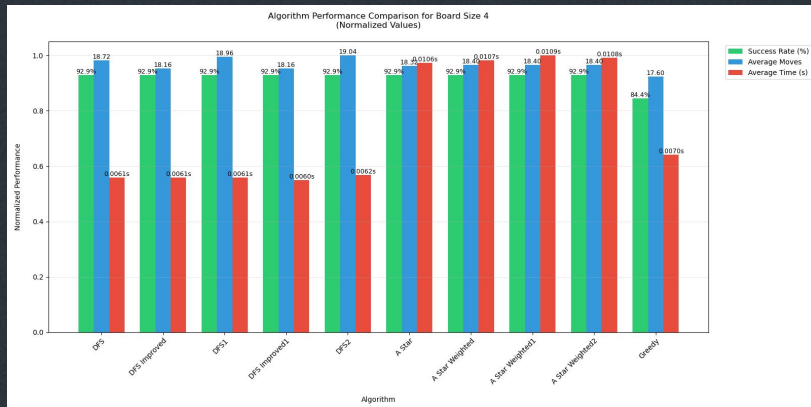
Where:

- b = branching factor
- d = depth limit
- h = heuristic depth
- w = weight

Experimental Results

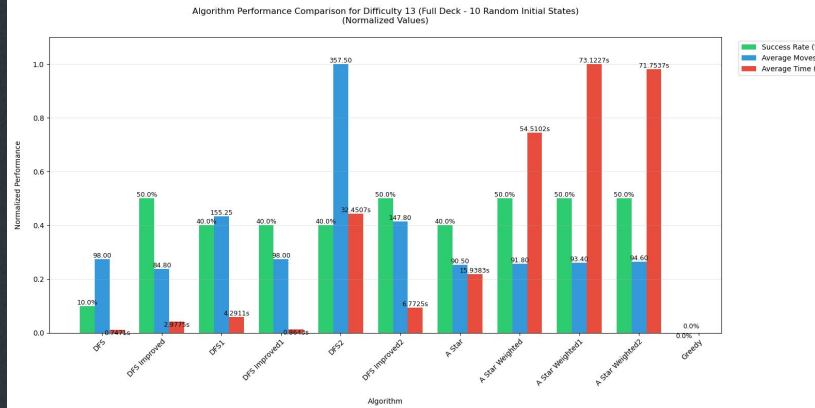
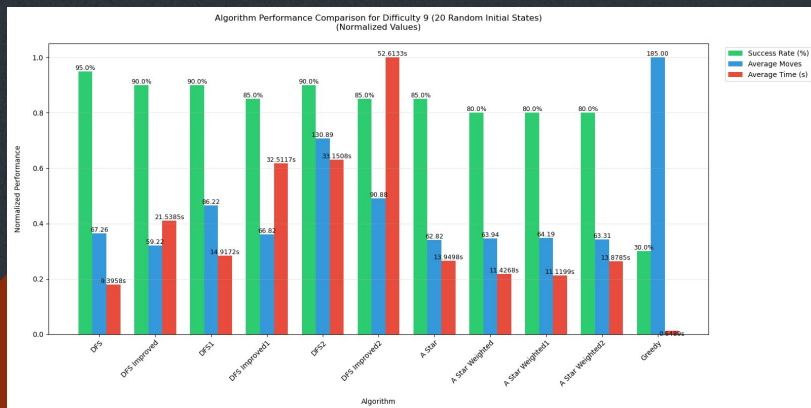
Board Size 4

Board Size 7



Board Size 9

Board Size 13 - Full Deck



Conclusions

- **DFS vs. DFS Improved:**
Standard DFS struggles with deep recursion and frequent useless moves on complex puzzles, while DFS Improved—by ordering successors to reduce the tableau—yields higher success rates and fewer moves, especially at low to moderate difficulties.
Increasing the depth limit provides higher success on hard cases but is also significantly slower.
- **Iterative Deepening:**
Thorough in theory but inefficient for Baker's Dozen since solutions require many moves, causing an explosion in search depth.
- **A* and Weighted A*:**
A* guarantees optimality, but Weighted A* (with proper weighting) offers faster runtimes at a slight loss of optimality. Both achieve high success rates on tougher deals but may use more resources than DFS on simpler puzzles.
- **Greedy:**
Extremely fast and good in terms of memory, yet often incomplete—good for quick progress in easy cases, but unreliable for complex deals.
- Further work would involve heuristics refinement, which is complex in the scope of this project, given the high complexity of Baker's Dozen Solitaire.

References & Materials

1. **Official Rules:** [Solitaire.org](http://solitaire.org) Bakers Dozen.
2. **Existing Implementations:**
 - Open-source Python solitaire frameworks (e.g., [pygame](#)).
 - Solitaire [Tutorial](#) in Python.
3. **Search Algorithms:**
 - Russell & Norvig, *Artificial Intelligence: [A Modern Approach](#)*.
 - Online Resources & Lecture Slides